Guía del notebook `1_07_descenso_por_gradiente`: entrenar una neurona completa

Puntos: 3 · **Tiempo:** 25–30 min · **Clase:** 4 · **Archivo:** `notebooks/1_07_descenso_por_gradiente.ipynb`
Teoría completa: Guía de la Clase 4, sección B3.

¿Qué vas a aprender?

- A entrenar una neurona con **todos** los ejemplos, durante muchas **épocas**.
- A inicializar los pesos con **Xavier**.
- A medir la **exactitud** con datos que la red no vio al entrenar.

Antes de empezar

1. `conda activate nni`.
2. VS Code → **Open Folder** → `curso-redes-neuronales` → `notebooks/1_07_descenso_por_gradiente.ipynb` → kernel **nni**.
   Usa la carpeta del repo: el notebook carga `data.csv`, `data_prep.py` y el paquete `nni` de la carpeta `notebooks`.
3. Ejecuta las celdas en orden con **Shift + Enter**.

La idea en 2 minutos

- **Datos:** 400 alumnos que pidieron entrar a un posgrado.
  - Entradas: puntaje GRE, promedio GPA y el prestigio de su universidad (`rank`, en 4 columnas de 0/1). Son **6 entradas**.
  - Salida: `admit`, que vale 1 si fue admitido.
  - `data_prep.py` ya los escala y separa **360 para entrenar** y **40 para probar**.
- **Algoritmo por época:**
  1. Para cada alumno: predicción $\hat{y}$, término de error $\delta = (y - \hat{y})\cdot\hat{y}(1 - \hat{y})$, y **acumular** $\delta \cdot x$.
  2. Al final de la época: pesos += η · acumulado / número de alumnos.
- **Xavier:** pesos al azar entre $-1/\sqrt{n}$ y $+1/\sqrt{n}$ ($n = 6$ → entre −0.408 y +0.408).
- **Exactitud** = aciertos / predicciones. La red dice "admitido" si $\hat{y} > 0.5$.

Paso a paso

Celda 1: paquetes y datos

Carga todo y fija la semilla 42. Si da error, revisa que estés en la carpeta del repositorio.

Celda `TODO (1 punto)`: inicializar con Xavier

Pista: `np.random.uniform(-1/np.sqrt(n_features), 1/np.sqrt(n_features), size=n_features)`.

Celda: exactitud antes de entrenar

✅ **Comprobación:** con la semilla 42 sale **0.650**. Es casi lo mismo que adivinar "no admitido" siempre.

Celda: hiperparámetros

`epochs = 10` y `learnrate = 0.5`. Más adelante subirás las épocas.

Celda `TODO (1 punto)`: el ciclo de entrenamiento

Completa los cuatro huecos siguiendo el algoritmo:

1. `output` → `nodo.forward(x)`.
2. `error_term` → (y − output) · output · (1 − output).
3. `incremento_w +=` → error_term · x.
4. Después del `for` de ejemplos: `weights +=` tasa · incremento acumulado, **dividido entre** `n_records`.

⚠️ Escribe `weights += ...` y no `weights = weights + ...`. La neurona `nodo` comparte el arreglo `weights`: con `+=` lo modificas "en el lugar" y la neurona ve el cambio. Con `=` creas un arreglo nuevo, la neurona se queda con los pesos viejos y **la pérdida no baja**.

Celda: gráfica de la pérdida

Debe bajar época tras época.

Celda: exactitud final (1 punto si supera 0.74)

✅ **Comprobación** (semilla 42, `learnrate = 0.5`):

Épocas	Pérdida al final	Exactitud
10	≈ 0.225	≈ 0.60
100	≈ **0.2055**	**0.775** ✓
1000	≈ 0.197	≈ 0.725

Para pensar

¿Por qué con 1000 épocas la pérdida baja pero la exactitud empeora? El conjunto de prueba es pequeño (40 alumnos) y la red se ajusta demasiado a los datos de entrenamiento: es **sobreajuste**.

Errores frecuentes

Síntoma	Causa	Solución
La pérdida no baja nada	`weights = weights + ...`	Usa `weights += ...`
`ModuleNotFoundError: data_prep` o `nni`	Notebook abierto fuera de la carpeta	Abre la carpeta `curso-redes-neuronales`
`ValueError: shapes not aligned`	`size` distinto de `n_features`	`size=n_features`
`WARNING - La pérdida ha aumentado`	Tasa muy grande	Baja `learnrate` a 0.1

Síntoma	Causa	Solución
Exactitud distinta a la tabla	No ejecutaste la celda con `np.random.seed(42)`	Ejecuta todo desde la primera celda